第3章: React入門

> React Native の土台は **React（リアクト）** です。Reactの「コンポーネント（部品）」「props（プロップス）」「state（ステート）」という3つの考え方を理解すれば、スマホアプリの画面づくりがぐっと楽になります。この章では完全初心者向けに、これらを順番に解説します。
>
> **この章のコードについて:** Reactの考え方を説明するため、一部Web向けの `<div>` などが出てきますが、**React Nativeでは `<View>` などに置き換わるだけで考え方は完全に同じ**です（対応は第4章で表にします）。まずは"考え方"に集中してください。

---

1. React とは — 画面を「部品」で組み立てる

React は、画面を **コンポーネント（component＝部品）** という小さなパーツに分けて作るための技術（ライブラリ）です。Meta社（旧Facebook）が開発しました。

> **身近な例え（レゴブロック）:** 小さなブロック（コンポーネント）を組み合わせて、家や車（画面全体）を作るイメージです。一度作ったブロックは別の場所でも再利用できます。「書籍カード」というブロックを1つ作れば、一覧画面で何個でも使い回せます。
>
> **「ライブラリ（library）」とは？** 「特定の機能を提供するプログラムの部品集」のこと。図書館（library）から本を借りるように、便利な機能を借りて使えます。Reactは「画面づくりの機能」を提供するライブラリです。

---

2. コンポーネント — 画面の部品

2.1 一番シンプルなコンポーネント

コンポーネントは「**何かの見た目を返す関数**」です。第2章で学んだ関数の知識がそのまま使えます。

コピー

```
// この一行は「React Nativeから Text という部品を借りてくる」という意味（詳細は第4章）
import { Text } from "react-native";

// Greeting : コンポーネント名。コンポーネント名は必ず大文字始まりにする決まり
// () => { ... } : 第2章で学んだアロー関数。引数なしの関数
// この関数は「画面に表示する内容」を return で返す
function Greeting() {
  // return の後ろが「見た目」。<Text>...</Text> が画面に出る文字
  return <Text>こんにちは！</Text>;
}

// export default : この部品を「外のファイルから使えるように公開する」宣言
export default Greeting;
```

コンポーネント名はなぜ大文字始まり？ Reactは「小文字始まり＝HTMLの普通のタグ」「大文字始まり＝自作コンポーネント」と区別します。だから自作の部品は必ず `Greeting` のように大文字で始めます。これはルールなので守りましょう。

`import` と `export` とは？ - `import`（インポート）：他のファイルや部品集から「機能を借りてくる」命令。- `export`（エクスポート）：自分のファイルの機能を「外に公開する」命令。ファイル同士で部品を貸し借りするための仕組みです。

2.2 JSX — `<Text>こんにちは</Text>` の正体

`return` の後ろに書いた `<Text>こんにちは！</Text>` のような、HTMLに似た記法を **JSX（ジェイエスエックス）** と呼びます。「JavaScriptの中に、画面の見た目を直接書ける特別な記法」です。

コピー

```
// JSXの基本ルール
return (
  {/* <View> は「箱」。中に複数の部品をまとめられる（第4章で詳説） */}
  <View>
    {/* タグは <開始>中身</終了> の形 */}
    <Text>1冊目の本</Text>
    <Text>2冊目の本</Text>
  </View>
);
// 複数行のJSXは ( ) で囲む / 一番外側は1つのタグ（ここでは<View>）でまとめる必要がある
// {/* ... */} はJSXの中でのコメントの書き方
```

JSX内の `{ }`（中カッコ）— 変数を埋め込む: JSXの中で `{ }` を使うと、その中にTypeScriptの値や式を埋め込めます。

コピー

```
const name = "鈴木";
return <Text>こんにちは、{name}さん</Text>;
// { name } : 中カッコの中はTypeScriptの世界。変数 name の中身（"鈴木"）が文字列に埋め込まれる
// 画面表示: こんにちは、鈴木さん
```

3. props — 部品に「材料」を渡す

同じ「書籍カード」でも、表示する本のタイトルは1冊ごとに違います。このように**コンポーネントの外から渡す材料**を **props（プロップス、propertiesの略）** と呼びます。

コピー

```
import { View, Text } from "react-native";

// BookCard : 書籍カードのコンポーネント
// props（材料）として title と author を受け取る形を、型で定義する
type BookCardProps = {
  // 本のタイトル（文字列）
  title: string;
  // 著者名（文字列）
  author: string;
};

// ({ title, author }) : 受け取ったpropsから title と author を取り出す書き方（分割代入という）
// : BookCardProps : 「受け取るpropsはこの型ですよ」と宣言
function BookCard({ title, author }: BookCardProps) {
  return (
    <View>
      {/* 受け取った title を表示 */}
      <Text>{title}</Text>
      {/* 受け取った author を表示 */}
      <Text>{author}</Text>
    </View>
  );
}

export default BookCard;
```

このBookCardを使う側は、HTMLの属性のようにpropsを渡します。

コピー

```
// title= と author= に渡したい値を指定する。これがpropsとしてBookCardに届く
<BookCard title="リーダブルコード" author="Dustin Boswell" />
<BookCard title="達人プログラマー" author="David Thomas" />
// 同じBookCard部品を、違う材料(props)で2回使い回している！ これがコンポーネントの便利さ
```

> 「**分割代入（ぶんかつだいにゅう）**」とは？ `{ title, author }` のように、オブジェクトの中から必要な項目だけを取り出して変数にする書き方です。`props.title` と毎回書く代わりに、最初に `title` だけ取り出しておけば短く書けます。Reactでは非常によく使います。
>
> **propsは「読み取り専用」**: 子コンポーネントは受け取ったpropsを**書き換えてはいけません**。あくまで「親から渡された材料を表示するだけ」と考えてください。値を変化させたいときは、次に学ぶ **state** を使います。

---

4. state — 部品が持つ「変化する値」

4.1 state とは

state（ステート＝状態） は、コンポーネントが内部に持つ「**変化する値**」です。たとえば「検索ボックスに入力された文字」「ボタンが押された回数」「読込中かどうか」などです。propsが「外から渡される材料」なのに対し、stateは「自分の中で変わっていく値」です。

4.2 `useState` の使い方

stateを作るには `useState` **（ユーズ・ステート）** という機能（フック）を使います。

コピー

```
// Reactから useState を借りてくる
import { useState } from "react";
import { View, Text, Button } from "react-native";

function Counter() {
  // useState(0) : 初期値0のstateを作る
  // 戻り値は配列で [今の値, 値を変える関数] の2つ。分割代入で受け取る
  // count       : 今の値（最初は0）
  // setCount    : countを変更するための専用関数（名前はset+state名が慣習）
  const [count, setCount] = useState(0);

  return (
    <View>
      {/* 今のcountを表示 */}
      <Text>押した回数: {count}</Text>
      <Button
        // ボタンに表示する文字
        title="押す"
        // onPress : 押されたときの処理 / setCountでcountを+1する
        onPress={() => setCount(count + 1)}
      />
    </View>
  );
}
```

`useState` の戻り値が配列なのはなぜ？ `useState` は「今の値」と「値を変える関数」の2つをセットで返します。`const [count, setCount] = useState(0)` は、その2つを配列として受け取り、`count` と `setCount` という名前を付けているのです。名前は自由ですが、変更関数は `set + 変数名` にするのが慣習です。

4.3 stateを直接書き換えてはいけない

コピー

```
const [count, setCount] = useState(0);

// ❌ これはダメ！ 直接書き換えても画面は更新されない
count = count + 1;
// ⭕ 必ず専用の関数（setCount）を使う。これで画面も自動更新される
setCount(count + 1);
```

> **なぜ専用関数を使う必要があるの？** Reactは「stateが変わったら画面を描き直す」という仕組みで動いています。`setCount` を呼ぶことで初めてReactが「変化した！画面を更新しよう」と気づきます。直接 `count = ...` と書くと、Reactは変化に気づけず、画面が古いままになります。これは初心者がつまずきやすい超重要ポイントです。

---

5. フック（Hooks）— Reactの便利機能

`useState` のように `use` で始まる機能を **フック（Hooks）** と呼びます。コンポーネントに様々な能力を追加する道具です。代表的なものを紹介します（後の章で使います）。

フック	役割	主な登場章
`useState`	変化する値（state）を持つ	全章
`useEffect`	画面表示時などのタイミングで処理を実行する	第7章
`useRef`	描き直しに関係しない値の保持・部品の操作（フォーカスなど）	本章 §5.2
`useMemo`	重い計算の結果を覚えておく（メモ化）	本章 §5.3
`useCallback`	関数そのものを覚えておく（再描画の抑制）	本章 §5.4
`useContext`	アプリ全体で値を共有する	本章 §5.5
`useReducer`	複数の関連する値をまとめて管理する	本章 §5.6
カスタムフック	自作の `use〜` でよく使う処理を再利用	本章 §5.7
`useRouter` / `useLocalSearchParams`	画面遷移やパラメータの取得（Expo Router）	第6・8章

5.1 `useEffect` のさわり

`useEffect`（ユーズ・エフェクト）は「**特定のタイミングで処理を実行する**」フックです。代表的な使い方が「画面が最初に表示されたとき、サーバーからデータを取ってくる」処理です。

コピー

```
import { useEffect, useState } from "react";

function BookList() {
  // 本のリストをstateで持つ（最初は空の配列[]）
  const [books, setBooks] = useState([]);

  // useEffect(実行したい処理, 依存配列) の形
  useEffect(() => {
    // この中に「画面表示時に1回だけやりたい処理」を書く（例: データ取得）
    console.log("画面が表示されました");
  }, []);
  // 第2引数の [] : 「依存配列」。空の[]は「最初の1回だけ実行する」という意味になる

  // books.length : 配列の要素数（本の冊数）
  return <Text>本の数: {books.length}</Text>;
}
```

> **「依存配列（いそんはいれつ）」とは？** `useEffect` の2つ目の引数 `[]` のことです。「この中の値が変わったら処理を再実行する」という指定です。空の `[]` にすると「最初の1回だけ」になります。第7章で実際にSupabaseからデータを取得する際に詳しく使います。今は「画面表示時の処理は useEffect に書く」とだけ覚えればOKです。

5.2 `useRef` —「描き直しに関係しない値」を覚えておく

`useRef`（ユーズ・レフ）は、「**画面の描き直しとは関係なく、こっそり値を覚えておきたい**」ときに使うフックです。代表的な使い道は2つあります。①入力欄（`TextInput`）を**プログラムからフォーカス（カーソルを当てる）**したいとき、②タイマーのIDなど「**画面に出さないけれど後で使う値**」を保管したいときです。

> **いつ使う？ 普通のstateや変数と何が違う？** `useState` は「値が変わったら画面を描き直す」のに対し、`useRef` は「**値が変わっても画面を描き直さない**」点が決定的に違います。「画面に出す値」は `useState`、「画面に出さず裏で持っておくだけの値」は `useRef`、と覚えてください。また、普通の変数（`let x = ...`）は描き直しのたびに作り直されて値が消えてしまいますが、`useRef` の中身は**描き直しをまたいで保たれます**。

例A: `TextInput` にカーソルを当てる（フォーカス制御）

> **▼ このコードがやること（先に日本語で）**：入力欄とボタンを置き、ボタンを押すと**入力欄に自動でカーソルが移動して、キーボードが開く**例です。「あの入力欄を操作したい」という指名のために `useRef` で入力欄への"つなぎ"を用意し、`ref.current?.focus()` でその入力欄にカーソルを当てます。問い合わせフォームなどで「最初の入力欄に自動でカーソルを合わせたい」場面で使います。

コピー

```
import { useRef } from "react";
import { View, TextInput, Pressable, Text } from "react-native";

export default function FocusExample() {
  // useRef<TextInput>(null) : TextInput を指すための「つなぎ」を作る。最初は何も指していないので null
  const inputRef = useRef<TextInput>(null);

  return (
    <View>
      {/* ref={inputRef} : この入力欄を inputRef に結びつける（あとで指名できるようにする） */}
      <TextInput ref={inputRef} placeholder="ここに入力" />

      {/* 押すと inputRef が指す入力欄にカーソルを当てる */}
      <Pressable onPress={() => inputRef.current?.focus()}>
        <Text>入力欄にカーソルを当てる</Text>
      </Pressable>
    </View>
  );
}
```

▼ コードを1つずつ分解して解説

「ボタンで入力欄にカーソルを当てる」部分を、1つずつ見ていきましょう。

解説1: 入力欄への「つなぎ」を作る `useRef`

コピー

```
const inputRef = useRef<TextInput>(null);
```

- `useRef`（ユーズ・レフ）は、「**ある部品や値を指しておくための“つなぎ（参照）”を作る**」道具です。
- `useRef<TextInput>(null)` は「`TextInput` を指すつなぎを作る。最初は何も指していないので `null`（空っぽ）」という意味です。
- 作ったつなぎを `inputRef` という変数に入れておきます。この `inputRef` を通して、あとで入力欄を操作します。
- ここで覚えた値は**画面を描き直す原因になりません**。あくまで「裏でこっそり持っておく」ためのものです。

> **用語: 参照（reference／ref）** とは「あの部品はこれだよ」と指し示す“つなぎ”のこと。`useRef` で作ったつなぎを部品に結びつけると、その部品をコードから操作できるようになります。

解説2: 入力欄とつなぎを結びつける `ref={inputRef}`

コピー

```
<TextInput ref={inputRef} placeholder="ここに入力" />
```

- `ref={inputRef}` は、「**この入力欄を、さきほど作った `inputRef` に結びつける**」指定です。
- これで `inputRef` は「画面上のこの `TextInput` 」を指すようになります。以後、`inputRef` 経由でこの入力欄を操作できます。
- `placeholder` は未入力時に薄く表示される案内文です（第4章で学んだとおり）。

> **用語: `ref` （レフ）属性** は、部品に「つなぎ」を結びつけるための特別な指定。`ref={つなぎ}` と書くことで、その部品をコードから指名できるようになります。

解説3: カーソルを当てる `inputRef.current?.focus()`

コピー

```
<Pressable onPress={() => inputRef.current?.focus()}>
  <Text>入力欄にカーソルを当てる</Text>
</Pressable>
```

- `inputRef.current` （カレント＝今指しているもの）は、「**つなぎ `inputRef` が今実際に指している部品**」を表します。ここでは結びつけた `TextInput` 本体です。
- `.focus()` （フォーカス）は「**その入力欄にカーソルを当てて、キーボードを開く**」命令です。つまり `inputRef.current.focus()` で「あの入力欄にカーソルを当てる」になります。
- 間にある `?.` （オプショナルチェーン）は「**もし `current` が空っぽ（null）なら、何もしない**」という安全装置です。まだ結びついていない一瞬のエラーを防ぎます。

> **用語: `.current`** は「ref が今指している中身」を取り出す入り口。`?.` は「左側が空っぽなら処理を止める」安全な書き方で、`null` によるエラーを防ぎます。

例B: タイマーのIDを保管する

> **▼ このコードがやること（先に日本語で）:** 「開始」ボタンで1秒ごとに数を数え始め、「停止」ボタンで止めるカウンターです。タイマーには「あとで止めるための番号（ID）」が発行されるので、それを `useRef` に保管しておき、停止時にその番号を使ってタイマーを止めます。「画面には出さないけれど後で使う値」を覚えておく、という `useRef` の使い方の例です。

コピー

```
import { useRef, useState } from "react";
import { View, Text, Pressable } from "react-native";

export default function TimerExample() {
  // 画面に出す数 → useState
  const [count, setCount] = useState(0);
  // タイマーのID（画面には出さない裏の値）→ useRef に保管。最初はまだ無いので null
  const timerRef = useRef<ReturnType<typeof setInterval> | null>(null);

  const start = () => {
    // すでに動いていたら二重に始めない
    if (timerRef.current) return;
    // setInterval : 一定間隔で処理を繰り返す。戻り値の「ID」を timerRef に保管
    timerRef.current = setInterval(() => {
      // 1秒ごとにカウントを1増やす
      setCount((prev) => prev + 1);
    }, 1000);
  };

  const stop = () => {
    if (timerRef.current) {
      // 保管しておいたIDでタイマーを止める
      clearInterval(timerRef.current);
      // 止めたのでIDを空に戻す
      timerRef.current = null;
    }
  };

  return (
    <View>
      <Text>カウント: {count}</Text>
      <Pressable onPress={start}><Text>開始</Text></Pressable>
      <Pressable onPress={stop}><Text>停止</Text></Pressable>
    </View>
  );
}
```

▼ コードを1つずつ分解して解説

「タイマーIDを覚えておいて、あとで止める」部分を、1つずつ見ていきましょう。

解説1: 画面に出す値と出さない値を分ける

コピー

```
// 画面に出す数 → useState
const [count, setCount] = useState(0);
const timerRef = useRef<ReturnType<typeof setInterval> | null>(null);
```

- `count` は**画面に表示する数**なので `useState` で持ちます。値が変わると画面が描き直され、新しい数が表示されます。
- `timerRef` は**タイマーのID**を入れるための入れ物です。これは画面に出さない裏の値なので `useRef` を使います。もし `useState` にすると、IDが変わるたびに無意味な描き直しが起きてしまいます。
- `ReturnType<typeof setInterval>` は「`setInterval` が返すID（番号）の型」を表す書き方です。今は「タイマーIDの型」とだけ理解すればOKです。

> **用語: `setInterval`（セット・インターバル）** は「決まった間隔で処理を繰り返す」命令で、止めるためのID（番号）を返します。このIDがないと、あとでタイマーを止められません。

解説2: タイマーを始めてIDを保管する `timerRef.current = ...`

コピー

```
const start = () => {
  // すでに動いていたら二重に始めない
  if (timerRef.current) return;
  timerRef.current = setInterval(() => {
    // 1秒ごとにカウントを1増やす
    setCount((prev) => prev + 1);
  }, 1000);
};
```

- `setInterval(処理, 1000)` は「**1000ミリ秒（＝1秒）ごとに、中の処理を繰り返す**」命令です。
- その戻り値（タイマーのID）を `timerRef.current` に代入して**保管**します。`useRef` の中身は `.current` に出し入れします。
- `if (timerRef.current) return;` は「すでにIDが保管されている（＝もう動いている）なら、何もせず終わる」という二重起動の防止です。
- `setCount((prev) => prev + 1)` は「**1つ前の値 `prev` に1を足す**」書き方で、繰り返し処理の中で安全にカウントを増やせます。

> **用語: `.current` への代入** は、ref の中身を入れ替える操作。`timerRef.current = id` で「このタイマーIDを覚えておいて」という意味になります。描き直しは起きません。

解説3: 保管したIDでタイマーを止める `clearInterval`

コピー

```
const stop = () => {
  if (timerRef.current) {
    // 保管しておいたIDでタイマーを止める
    clearInterval(timerRef.current);
    // 止めたのでIDを空に戻す
    timerRef.current = null;
  }
};
```

- `clearInterval(ID)`（クリア・インターバル）は「**そのIDのタイマーを止める**」命令です。止めるには、開始時に保管しておいたIDが必要です。
- だからこそ、開始時に `timerRef` へIDを保管しておいたのです。`useRef` のおかげで、描き直しをまたいでもIDが保たれています。
- 止めたあとは `timerRef.current = null` で「もうタイマーは無い」状態に戻します。これで再び「開始」を押せるようになります。

> **用語:** `clearInterval` は `setInterval` で始めたタイマーを止める命令。「始めたら止める」がセットで、止めるためのIDの保管に `useRef` がぴったりです。

5.3 `useMemo` — 重い計算の結果を「覚えておく」

`useMemo`（ユーズ・メモ）は、「**時間のかかる計算の結果を覚えておき、必要なときだけ計算し直す**」フックです。たとえば「たくさんの本の中から条件に合うものだけ絞り込む」「並べ替える」といった計算を、毎回やり直さずに済ませられます。

> **いつ使う？ 普通の変数と何が違う？** 普通に `const result = 重い計算()` と書くと、画面が描き直されるたびに**毎回計算し直され**ます。`useMemo` を使うと「**指定した値が変わったときだけ**計算し直し、それ以外は前回の結果を使い回す」ようになります。「結果は同じなのに毎回計算するのは無駄」というときの節約の道具です。最初は無理に使わず、「動作が重いと感じたら検討する」くらいでOKです。
>
> **▼ このコードがやること（先に日本語で）**：入力した言葉で本のリストを**絞り込み（検索）**ます。検索の絞り込み計算を `useMemo` で覚えておき、**検索の言葉が変わったときだけ**計算し直します。リストが大きいほど効いてくる「無駄な計算を減らす」工夫の例です。各部分の意味はコード内コメントにあります。

コピー

```
import { useMemo, useState } from "react";
import { View, TextInput, FlatList, Text } from "react-native";

const books = [
  { id: "1", title: "リーダブルコード" },
  { id: "2", title: "達人プログラマー" },
  { id: "3", title: "TypeScript入門" },
];

export default function BookSearch() {
  // 検索の言葉（画面に出す値なので useState）
  const [keyword, setKeyword] = useState("");

  // useMemo(() => 計算, [keyword]) : keyword が変わったときだけ絞り込みをやり直す
  const filtered = useMemo(() => {
    // includes : 文字列の中にその言葉が含まれるか判定。含む本だけ残す
    return books.filter((book) => book.title.includes(keyword));
  }, [keyword]);

  return (
    <View>
      <TextInput
        placeholder="本を検索"
        value={keyword}
        onChangeText={setKeyword}
      />
      <FlatList
        // 絞り込んだ結果を表示
        data={filtered}
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => <Text>{item.title}</Text>}
      />
    </View>
  );
}
```

▼ コードを1つずつ分解して解説

「検索の言葉が変わったときだけ絞り込みをやり直す」部分を、1つずつ見ていきましょう。

解説1: 計算結果を覚えておく `useMemo`

コピー

```
const filtered = useMemo(() => {
  return books.filter((book) => book.title.includes(keyword));
}, [keyword]);
```

- `useMemo(() => 計算, 依存配列)` の形で使います。`() => { ... }` の中に「**覚えておきたい計算**」を書き、その結果が `filtered` に入ります。
- `books.filter(...)` は「配列の中から条件に合うものだけ残す」計算です。ここでは「タイトルに `keyword` を含む本だけ」を残しています。
- `book.title.includes(keyword)` の `includes`（インクルーズ＝含む）は「その文字列に `keyword` が含まれているか」を判定します。

> **用語: メモ化（memoization）** とは「一度計算した結果を覚えておいて、同じ条件のときは計算し直さず再利用する」工夫のこと。`useMemo` の “Memo” はこの意味です。

解説2: 計算をやり直す条件 `[keyword]`（依存配列）

コピー

```
}, [keyword]);
```

- 最後の `[keyword]` が**依存配列**です。「**この中の値が変わったときだけ、上の計算をやり直す**」という指定です（`useEffect` の依存配列と同じ考え方）。
- ここでは `keyword`（検索の言葉）が変わったときだけ絞り込みをやり直し、それ以外の描き直しでは**前回の結果をそのまま使い回します**。
- たとえば検索とは関係ない別のstateが変わって画面が描き直されても、`keyword` が同じなら絞り込み計算はスキップされます。これが「無駄な計算を減らす」効果です。

> **用語: 依存配列** は「ここに書いた値が変わったら計算し直す」というリスト。`useMemo` でも `useEffect` でも、この配列で「やり直すタイミング」を指定します。

5.4 `useCallback` — 「関数そのもの」を覚えておく

`useCallback`（ユーズ・コールバック）は、`useMemo` の「関数版」です。「**毎回作り直されてしまう関数を、同じものとして覚えておく**」フックです。とくに `FlatList` の `renderItem` のように「子の部品に渡す関数」を安定させたいときに役立ちます。

いつ使う？ 何が違う？ コンポーネントの中で `const f = () => {...}` と書くと、画面が描き直されるたびに**新しい関数が作り直され**ます。たいていは問題ありませんが、その関数を子の部品（とくに最適化された部品）に渡していると、「中身は同じなのに別物の関数が来た」と判断されて**子も無駄に描き直される**ことがあります。`useCallback` で関数を覚えておくと、それを防げます。`useMemo` が「値（計算結果）」を覚えるのに対し、`useCallback` は「関数そのもの」を覚える、と整理してください。

▼ このコードがやること（先に日本語で）：本のリストを `FlatList` で表示し、**1件をどう描くかの関数（`renderItem`）を `useCallback` で安定させる**例です。こうすると、関係のない描き直しが起きても `renderItem` が作り直されず、リストの無駄な再描画を抑えられます。大きなリストでカクつきを減らしたいときの工夫です。各部分の意味はコード内コメントにあります。

コピー

```
import { useCallback } from "react";
import { FlatList, View, Text } from "react-native";

const books = [
  { id: "1", title: "リーダブルコード" },
  { id: "2", title: "達人プログラマー" },
];

export default function StableList() {
  // useCallback(関数, 依存配列) : この関数を「同じもの」として覚えておく
  // 依存配列が [] なので、最初に作った関数をずっと使い回す
  const renderItem = useCallback(
    ({ item }: { item: { id: string; title: string } }) => (
      <View>
        <Text>{item.title}</Text>
      </View>
    ),
    []
  );

  return (
    <FlatList
      data={books}
      keyExtractor={(item) => item.id}
      // 安定した関数を渡す
      renderItem={renderItem}
    />
  );
}
```

▼ コードを1つずつ分解して解説

「`renderItem` を作り直さないようにする」部分を、1つずつ見ていきましょう。

解説1: 関数を覚えておく `useCallback`

コピー

```
const renderItem = useCallback(
  ({ item }: { item: { id: string; title: string } }) => (
    <View>
      <Text>{item.title}</Text>
    </View>
  ),
  []
);
```

- `useCallback(関数, 依存配列)` の形で使います。第1引数に「**覚えておきたい関数**」を渡すと、その関数が `renderItem` に入ります。
- ここで渡しているのは「1件分（`item`）を受け取って、そのタイトルを表示する関数」です。`FlatList` の `renderItem` として使う形と同じです。
- 普通に書くと描き直しのたびに新しい関数が作られますが、`useCallback` で包むことで「**前と同じ関数**」として扱われます。

> **用語: `useCallback`** は「関数そのものをメモ化する」フック。`useMemo` が計算結果を覚えるのに対し、こちらは関数を覚えて、毎回の作り直しを防ぎます。

解説2: 作り直す条件 `[]`（依存配列）

コピー

```
  [],
```

- 最後の `[]` は**依存配列**です。「**この中の値が変わったときだけ関数を作り直す**」という指定です。
- ここでは空の `[]` なので「**最初に作った関数をずっと使い回す**」という意味になります。`renderItem` は中身が変わらないので、これでOKです。
- もし関数の中で `keyword` などの値を使うなら、`[keyword]` のようにその値を入れます。そうしないと「古い値を覚えたままの関数」になってしまうので注意します。

> **用語: 依存配列** は `useMemo`・`useEffect` と同じく「作り直すタイミング」を決めるリスト。空の `[]` は「ずっと同じものを使い回す」という意味です。

5.5 `useContext` — アプリ全体で値を共有する

`useContext`（ユーズ・コンテキスト）は、「**アプリ全体（または広い範囲）で同じ値を共有する**」ためのフックです。たとえば「ダークモードかどうか（テーマ）」「表示言語」「ログイン中のユーザー」など、たくさんの画面が共通して使う値に向いています。

> **いつ使う？ propsと何が違う？** 普通は親から子へ値を渡すには `props` を使いますが、深い階層まで届けるには「親→子→孫→ひ孫…」と**毎段props を手渡し**しなければなりません（これを"バケツリレー"と呼びます）。
> `useContext` を使うと、その手渡しを飛ばして「**どの階層からでも直接、共有の値を取り出せる**」ようになります。「多くの場所で使う、アプリ共通の値」に向いた道具です。
>
> **▼ このコードがやること（先に日本語で）**：「テーマ（色のモード）」をアプリ全体で共有し、**離れた場所にある部品から、props を経由せずに直接そのテーマを取り出して使う**例です。共有したい値の"置き場所"を作り（Context）、いちばん外側でその値を配り、使いたい部品が `useContext` で受け取ります。各ステップの意味はコード内コメントにあります。

コピー

```
import { createContext, useContext } from "react";
import { View, Text } from "react-native";

// 1. 共有したい値の「置き場所」を作る。初期値は "light"（明るいテーマ）
const ThemeContext = createContext("light");

// 3. 共有された値を「使う」部品（離れた場所にあってもOK）
function ThemeLabel() {
  // useContext : 共有されたテーマを直接受け取る
  const theme = useContext(ThemeContext);
  return <Text>今のテーマ: {theme}</Text>;
}

// 2. いちばん外側で値を「配る」。value で配りたい値を指定
export default function App() {
  return (
    <ThemeContext.Provider value="dark">
      <View>
        {/* ThemeLabel には props を渡していないのに、テーマを受け取れる */}
        <ThemeLabel />
      </View>
    </ThemeContext.Provider>
  );
}
```

▼ コードを1つずつ分解して解説

「props を使わずにアプリ全体で値を共有する」流れを、1つずつ見ていきましょう。

解説1: 値の置き場所を作る `createContext`

コピー

```
const ThemeContext = createContext("light");
```

- `createContext(初期値)`（クリエイト・コンテキスト）は、「**共有したい値の"置き場所"を作る**」命令です。
- `"light"` は初期値で、「もし値が配られていなければ、とりあえずこれを使う」という保険です。
- 作った置き場所を `ThemeContext` という名前にしました。以後、この名前を通して値を配ったり受け取ったりします。

> **用語: Context（コンテキスト＝文脈・背景）** とは「アプリ全体で共有する値の置き場所」のこと。ここに値を置いておくと、どの部品からでも取り出せるようになります。

解説2: いちばん外側で値を配る `Provider`

コピー

```
<ThemeContext.Provider value="dark">
  <View>
    <ThemeLabel />
  </View>
</ThemeContext.Provider>
```

- `ThemeContext.Provider`（プロバイダー＝提供者）で囲んだ範囲に、`value` で指定した値が**配られます**。ここでは `value="dark"` なので、囲んだ中の部品はみな「`dark`」というテーマを受け取れます。
- 囲まれている `<ThemeLabel />` には **props を1つも渡していません**。それでもテーマを受け取れるのが Context の特徴です。
- ふつうはアプリの**いちばん外側**でこの `Provider` を置き、アプリ全体に値を配ります。

> **用語: Provider（プロバイダー）** は「Context の値を配る役」。`<...Provider value={値}>` で囲んだ範囲の部品すべてに、その値が届きます。

解説3: どこからでも値を受け取る `useContext`

コピー

```
function ThemeLabel() {
  // useContext : 共有されたテーマを直接受け取る
  const theme = useContext(ThemeContext);
  return <Text>今のテーマ: {theme}</Text>;
}
```

- `useContext(ThemeContext)` は、「**`ThemeContext` に配られている値を、直接受け取る**」フックです。受け取った値を `theme` に入れています。
- `ThemeLabel` は親から props をもらっていませんが、`useContext` のおかげで「`dark`」というテーマを取り出せます。
- これにより、親→子→孫…と props を手渡しする"バケツリレー"をしなくても、必要な部品が**直接**共有の値を使えます。

> **用語: バケツリレー（prop drilling）** とは、深い階層へ props を1段ずつ手渡しすること。`useContext` はこれを飛ばして、必要な場所で直接値を受け取れるようにします。

5.6 `useReducer` — フォームの複数の値をまとめて管理する

`useReducer`（ユーズ・リデューサー）は、「**たくさんの関連する値を、1か所のルールでまとめて管理する**」フックです。`useState` の発展形といえます。たとえば「本のタイトル・著者・メモ」をまとめて扱うフォームのように、複数の値が連動する場面で力を発揮します。

> **いつ使う？ useStateと何が違う？** `useState` は値1つにつき1つ作るので、フォームの項目が増えると `useState` がいくつも並んで管理が大変になります。`useReducer` は「**1つのまとまった state**」と「**それをどう変えるかのルール（reducer）**」をセットにして、更新の処理を1か所に集約できます。項目が多いフォームや、「複数の値が一緒に変わる」ような複雑な状態では `useReducer` が見通しよくなります。少数の単純な値なら `useState` のままでOKです。
>
> **▼ このコードがやること（先に日本語で）**：「タイトル」と「著者」の2つの入力欄を持つフォームを、**1つのまとまった状態として管理する**例です。「どの項目を、どんな値に変えるか」という指示を1か所のルール（reducer）に集約し、入力のたびにその指示を送って状態を更新します。項目が増えても管理しやすい形です。各部分の意味はコード内コメントにあります。

コピー

```
import { useReducer } from "react";
import { View, TextInput, Text } from "react-native";

// 1. 管理する状態のまとまり（最初の値）
const initialState = { title: "", author: "" };

// 2. reducer ： 「今の状態」と「指示(action)」を受け取り、「新しい状態」を返すルール
type Action = { field: "title" | "author"; value: string };
function reducer(state: typeof initialState, action: Action) {
  // ...state ： 今の状態をそのままコピーし、指定された項目だけ上書きする
  return { ...state, [action.field]: action.value };
}

export default function BookForm() {
  // useReducer(ルール, 最初の状態) ：[今の状態, 指示を送る関数] が返る
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <View>
      <TextInput
        placeholder="タイトル"
        value={state.title}
        // dispatch ： reducer に「titleをこの値にして」と指示を送る
        onChangeText={(text) => dispatch({ field: "title", value: text })}
      />
      <TextInput
        placeholder="著者"
        value={state.author}
        onChangeText={(text) => dispatch({ field: "author", value: text })}
      />
      <Text>入力中: {state.title} / {state.author}</Text>
    </View>
  );
}
```

▼ コードを1つずつ分解して解説

「複数の入力を1か所のルールでまとめて管理する」流れを、1つずつ見ていきましょう。

解説1: 管理する状態のまとまり `initialState`

コピー

```
const initialState = { title: "", author: "" };
```

- `initialState`（イニシャル・ステート＝最初の状態）は、「**まとめて管理したい値の初期状態**」です。

- ここでは `title`（タイトル）と `author`（著者）の2つを1つのオブジェクトにまとめ、どちらも最初は空文字 `""` にしています。
- `useState` のように値ごとにバラバラに持つのではなく、**関連する値を1つのまとまりにする**のが `useReducer` の考え方です。

> **用語: 状態のまとまり** とは、関連する複数の値を1つのオブジェクトにまとめたもの。フォームのように「一緒に扱いたい値」を1か所に集めると管理しやすくなります。

解説2: 状態の変え方を決めるルール `reducer`

コピー

```
type Action = { field: "title" | "author"; value: string };
function reducer(state: typeof initialState, action: Action) {
  return { ...state, [action.field]: action.value };
}
```

- `reducer`（リデューサー）は、「**今の状態（`state`）と指示（`action`）を受け取り、新しい状態を返す**」ルールの関数です。状態を変える処理を、この1か所に集約します。
- `action` は「どの項目（`field`）を、どんな値（`value`）にするか」という指示です。たとえば `{ field: "title", value: "本A" }` なら「タイトルを“本A”に」という意味です。
- `{ ...state, [action.field]: action.value }` は「**今の状態を丸ごとコピーし、指示された項目だけ上書きする**」書き方です。`...state` がコピー、`[action.field]: action.value` が上書き部分です。

> **用語: reducer（リデューサー）** とは「今の状態＋指示 → 新しい状態」を計算するルールの関数。状態の変え方を1か所にまとめることで、更新処理が散らからずに済みます。

解説3: 状態と「指示を送る関数」を受け取る `useReducer`

コピー

```
const [state, dispatch] = useReducer(reducer, initialState);
```

- `useReducer(ルール, 最初の状態)` は、`[今の状態, 指示を送る関数]` の2つを配列で返します（`useState` が `[値, 変える関数]` を返すのと似た形です）。
- 「今の状態」を `state`、「指示を送る関数」を `dispatch`（ディスパッチ＝送る）という名前で受け取っています。
- これ以降、状態を直接いじるのではなく、**`dispatch` で指示を送る**ことで `reducer` が新しい状態を作ってくれます。

> **用語: `dispatch`（ディスパッチ）** は「reducer に指示を送る関数」。`dispatch(指示)` を呼ぶと、reducer がその指示にしたがって状態を更新します。

解説4: 入力のたびに指示を送る `dispatch(...)`

コピー

```
<TextInput
  placeholder="タイトル"
  value={state.title}
  onChangeText={(text) => dispatch({ field: "title", value: text })}
/>
```

- `value={state.title}` で、入力欄の中身を状態の `title` に結びつけています（第4章で学んだ制御コンポーネントの形）。
- `onChangeText` で、入力が変わるたびに `dispatch({ field: "title", value: text })` を呼びます。これは「**タイトルを、いま入力された `text` に変えて**」という指示です。
- この指示を受け取った `reducer` が新しい状態を作り、画面が更新されます。著者欄も同じ仕組みで、`field` を `"author"` にするだけです。
- 項目が増えても「入力欄を足して、`field` を変えて `dispatch` する」だけなので、`useState` をいくつも並べるより見通しよく管理できます。

> **用語: action（アクション＝指示）** とは「状態をどう変えてほしいか」を表すデータ。`dispatch(action)` で送ると、reducer がそれに応じて状態を更新します。

5.7 カスタムフック — 自分でフックを作る

ここまで紹介したのはReactが用意したフックですが、**自分で `use〜` という関数を作って、よく使う処理をまとめる**こともできます。これを **カスタムフック** と呼びます。同じような state の操作を何度も書くなら、1つのフックにまとめて使い回せます。

> **いつ使う？ ふつうの関数と何が違う？** カスタムフックは「**中で `useState` などのフックを使う、再利用できる処理のかたまり**」です。ふつうの関数と違い、中でフックを呼べるのが特徴です。「複数の画面で同じ state の扱い方をする」なら、その部分をカスタムフックに切り出すと、各画面はそれを1行呼ぶだけで済みます。名前は必ず `use` で始めるのが決まりです。
>
> **▼ このコードがやること（先に日本語で）**：「オン／オフを切り替えるだけ」という、よくある処理を `useToggle` という自作のフックにまとめ、それを画面から使う例です。フックの中に「今の状態」と「切り替える処理」をまとめておけば、使う側はそれを受け取って呼ぶだけで済みます。同じ処理をいろいろな画面で使い回せるのが利点です。各部分の意味はコード内コメントにあります。

コピー

```
import { useState } from "react";
import { View, Text, Pressable } from "react-native";

// カスタムフック : 名前は必ず use で始める。中で useState を使える
// 「今オンかどうか」と「切り替える処理」をまとめて返す
function useToggle(initial: boolean) {
  const [on, setOn] = useState(initial);
  // !prev : 今の逆にする（オン⇔オフ）
  const toggle = () => setOn((prev) => !prev);
  // 状態と切り替え処理をまとめて返す
  return { on, toggle };
}

export default function ToggleExample() {
  // 自作フックを使う。useState のように [今の値, 操作] を受け取れる
  const { on, toggle } = useToggle(false);

  return (
    <View>
      {/* on が true なら「オン」 */}
      <Text>状態: {on ? "オン" : "オフ"}</Text>
      <Pressable onPress={toggle}>
        <Text>切り替える</Text>
      </Pressable>
    </View>
  );
}
```

▼ コードを1つずつ分解して解説

「よく使う処理を自作フックにまとめて使う」流れを、1つずつ見ていきましょう。

解説1: 自分でフックを作る `function useToggle`

コピー

```
function useToggle(initial: boolean) {
  const [on, setOn] = useState(initial);
  // !prev : 今の逆にする（オン⇔オフ）
  const toggle = () => setOn((prev) => !prev);
  // 状態と切り替え処理をまとめて返す
  return { on, toggle };
}
```

- `useToggle` は自作のフックです。**名前が `use` で始まっている**のがポイントで、これがフックである印です（カスタムフックは必ず `use` で始める決まり）。
- 中で `useState` を使って「今オンかどうか（`on`）」を持っています。ふつうの関数と違い、**カスタムフックの中ではフックを呼べます**。
- `toggle` は「**今の値の逆にする**」処理です。`!prev` の `!`（ビックリマーク）は「逆にする」記号で、`true` ↔ `false` を入れ替えます。
- 最後に `{ on, toggle }`（今の状態と切り替え処理）を**まとめて返し**ています。使う側はこれを受け取って使います。

> **用語: カスタムフック** とは「自分で作った `use〜` 関数」。中でフックを使える点がふつうの関数と違い、よく使う state の処理をまとめて使い回すために作ります。

解説2: 自作フックを使う

コピー

```
const { on, toggle } = useToggle(false);
```

- `useToggle(false)` で自作フックを呼び、「最初はオフ（`false`）の切り替え状態」を用意しています。
- 返ってきた `{ on, toggle }` を分割代入で受け取り、`on`（今の状態）と `toggle`（切り替え処理）を使えるようにしています。`useState` を使うときと似た感覚です。
- 画面側はこの**1行**を書くだけで、オン／オフの状態と切り替え処理が手に入ります。同じ処理を別の画面でも使いたければ、また `useToggle()` を呼ぶだけです。

> **用語: 再利用（さいりよう）** とは「一度作ったものを、いろいろな場所で使い回すこと」。カスタムフックにまとめておくと、同じ state の処理を各画面で何度も書かずに済みます。

解説3: 状態に応じて表示を変える `on ? "オン" : "オフ"`

コピー

```
{/* on が true なら「オン」 */}
<Text>状態: {on ? "オン" : "オフ"}</Text>
<Pressable onPress={toggle}>
  <Text>切り替える</Text>
</Pressable>
```

- `{on ? "オン" : "オフ"}` は「`on` が `true` なら『オン』、そうでなければ『オフ』を表示する」という書き方です（`条件 ? Aのとき : Bのとき` の形で、三項演算子と呼びます）。
- ボタンの `onPress={toggle}` は「押されたら、自作フックの `toggle`（切り替え処理）を実行する」指定です。

- 押すたびに `on` が反転し、表示も「オン」「オフ」と切り替わります。状態の管理と切り替え処理は、すべて `useToggle` の中にまとまっています。

> **用語: 三項演算子（さんこうえんざんし）** とは `条件 ? A : B` の形で「条件が成り立てばA、そうでなければB」を選ぶ書き方。JSXの中で表示を出し分けるときによく使います。

---

6. コンポーネントを組み合わせる

Reactの本質は「小さなコンポーネントを組み合わせて大きな画面を作る」ことです。先ほどのBookCardを使って、一覧画面を組み立ててみましょう。

コピー

```
import { View, Text } from "react-native";

// 子コンポーネント：1冊分のカード（再掲）
type BookCardProps = { title: string; author: string };
function BookCard({ title, author }: BookCardProps) {
  return (
    <View>
      <Text>{title}</Text>
      <Text>{author}</Text>
    </View>
  );
}

// 親コンポーネント：一覧画面。BookCardを並べて使う
function BookListScreen() {
  return (
    <View>
      <Text>書籍一覧</Text>
      {/* 同じBookCardを、異なるpropsで何度も並べる */}
      <BookCard title="リーダブルコード" author="Dustin Boswell" />
      <BookCard title="達人プログラマー" author="David Thomas" />
      <BookCard title="TypeScript入門" author="鈴木 僚太" />
    </View>
  );
}

export default BookListScreen;
```

> **親子関係**: `BookListScreen` （親）が `BookCard` （子）を使っています。親が子にpropsで材料を渡す、という関係です。実際のアプリでは、本のデータは配列で管理し、第2章で学んだ `map` を使って自動的にカードを並べます（第7章で実装します）。

---

7. Web の React と React Native の違い（予告）

この章のコードは「考え方」を説明するためのもので、実際のReact Nativeでは部品の名前が少し変わります。次章への橋渡しとして、対応表を載せておきます。

役割	WebのReact（HTML）	React Native
箱・まとまり	`<div>`	`<View>`
文字の表示	`<p>`, `<span>`	`<Text>`
ボタン	`<button>`	`<Button>`, `<Pressable>`
入力欄	`<input>`	`<TextInput>`
画像	`<img>`	`<Image>`

> **重要: コンポーネント・props・state・フックという「考え方」は、WebでもReact Nativeでもまったく同じ**です。違うのは「使う部品の名前」と「文字は必ず `<Text>` で囲む」などのモバイル特有のルールだけです。次章でその違いを実際に手を動かして学びます。

---

8. この章のまとめ

- **コンポーネント**: 見た目を返す関数。名前は大文字始まり。これを組み合わせて画面を作る
- **JSX**: `<Text>...</Text>` のように見た目を書く記法。`{ }` で変数を埋め込める
- **props**: 部品の外から渡す「材料」。読み取り専用
- **state**: 部品が内部に持つ「変化する値」。`useState` で作り、変更は必ず `setXxx` 関数で
- **フック**: `use` で始まる便利機能。`useState` （状態）、`useEffect` （タイミング処理）など
- これらの考え方は **WebでもReact Nativeでも共通**。違うのは部品の名前

> **次の章へ**: いよいよReact Native本体です。第4章では、`View` / `Text` / `FlatList` などスマホ専用の部品と、画面を配置するFlexbox、そしてExpo Routerによる画面遷移を学びます。